MAKÜ
BURDUR MEHMET AKİF ERSOY ÜNİVERSİTESİ

VERİ YAPILARI VE ALGORİTMALAR

HAFTA 12
SIRALAMA ALGORİTMALARI

Bucak Bilgisayar ve Bilişim Fakültesi

Ders İçeriği: Hafta 12

- Sıralama Algoritmaları
  - Kabarcık Sıralama (Bubble Sort)
  - Seçmeli Sıralama (Selection Sort)
  - Eklemeli Sıralama (Insertion Sort)
  - Hızlı Sıralama (Quick Sort)
  - Birleştirmeli Sıralama (Merge Sort)
  - Sıralama Algoritmalarının Performansı

Sıralama Algoritmaları

- Sıralama Algoritmaları, elimizde var olan verileri belirli bir düzene veya bir sıraya koymak için geliştirilen bazı algoritmalardır.
- Bilgisayar Biliminde önemli bir yer tutup performansı etkileyebilmektedir.
- Büyükten küçüğe, küçükten büyüğe, permütasyon gibi birçok sıralama algoritması bulunur.

Kabarcık Sıralama Algoritması

- Bubble Sort
- Temel işleyiş mantığı, bir dizideki bütün ardışık iki elemanı karşılaştırarak, sıralamayı bozuyorsa yer değiştirmektir.
- İlk iterasyonda dizinin baştan sona ardışık iki elemanı karşılaştırılır, sıralamayı bozuyorsa yer değiştirilir. Böylece dizinin son elemanı sıralanmış olur. (Küçükten büyüğe sıralamada en büyük eleman sona alınmış olur.)
- Sonraki iterasyonda bu en sağdaki elemanı işleme koymaya gerek yoktur. Bunun için iç döngüdeki tekrar sayısı bir azaltılır.
- Son iterasyonda ise dizinin solunda kalan iki eleman da sıralanmakta ve sıralama işlemi tamamlanmaktadır.

Kabarcık Sıralama Algoritması

- Bubble Sort
- **Örnek:** 9, 5, 8, 3, 1 rakamlarının artan şekilde sıralanmasını kabarcık algoritmasıyla gerçekleştirelim.
  - 1. iterasyon: n-1 karşılaştırma

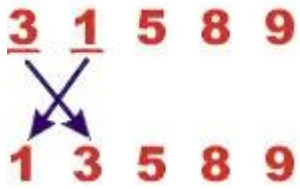
  - 1. iterasyon tamamlandığında en büyük eleman olan 9 en sona yerleşmiş olur ve bir daha karşılaştırmaya gerek yoktur.

Kabarcık Sıralama Algoritması

- 2. iterasyon: n-2 karşılaştırma

- 3. iterasyon: n-3 karşılaştırma

- 4. iterasyon: 1 karşılaştırma (son iterasyon)



Kabarcık Sıralama Algoritması

- Bubble Sort
- **Örnek:** Algoritmayı kullanarak aşağıdaki verileri sıralayalım.
  - Veriler : 25 57 48 37 12 92 86 33
  - İterasyon 1 : 25 48 37 12 57 86 33 **92** n-1 karşılaştırma

25 57 48 37 12 92 86 33

25 57 48 37 12 92 86 33

25 48 57 37 12 92 86 33

25 48 37 57 12 92 86 33

25 48 37 12 57 92 86 33

25 48 37 12 57 92 86 33

25 48 37 12 57 86 92 33

25 48 37 12 57 86 33 **92** **son eleman sıralandı**

Kabarcık Sıralama Algoritması

- Bubble Sort
- **Örnek:** Algoritmayı kullanarak aşağıdaki verileri sıralayalım.
  - Veriler : 25 57 48 37 12 92 86 33
  - İterasyon 1 : 25 48 37 12 57 86 33 **92** n-1 karşılaştırma
  - İterasyon 2 : 25 37 12 48 57 33 **86** 92 n-2 karşılaştırma
  - İterasyon 3 : 25 12 37 48 33 **57** 86 92 n-3 karşılaştırma
  - İterasyon 4 : 12 25 37 33 **48** 57 86 92 …
  - İterasyon 5 : 12 25 33 **37** 48 57 86 92 …
  - İterasyon 6 : 12 25 **33** 37 48 57 86 92 2 karşılaştırma
  - İterasyon 7 : 12 **25** 33 37 48 57 86 92 1 karşılaştırma

Kabarcık Sıralama Algoritması

- Bubble Sort
- Kodlar:

```
for (i = 0; i < n - 1; i++) {
    for (j = 0; j < n - i - 1; j++) {
        if (dizi[j] > dizi[j + 1]) {
            tmp = dizi[j];
            dizi[j] = dizi[j + 1];
            dizi[j + 1] = tmp;
        }
    }
}
```

Kabarcık Sıralama Algoritması

- Bubble Sort
- Bubble sort, sıralama teknikleri içinde anlaşılması ve programlanması kolaydır. Ama zaman karmaşıklığı nedeniyle büyük verilerde etkili değildir.

- Zaman Karmaşıklığı (Time Complexity):
  - En Kötü Durum (Worst Case): $O(n^2)$
  - Ortalama Durum (Average Case): $O(n^2)$
  - En İyi Durum (Best Case): $O(n)$
    - (Sıralı bir diziyi algılamak için **flag(bayrak)** mekanizması kullanılır.)
- Bellek Karmaşıklığı (Space Complexity): $O(1)$

Seçmeli Sıralama Algoritması

- Selection Sort
- Elemanların seçilerek uygun yerlerine konulması ile gerçekleştirilen bir sıralamadır.
- Dizideki ilk eleman ile en küçük eleman yer değiştirir.
  - En küçük elemanı bulmak için ilk elemandan son elemana kadar karşılaştırma yapılır.
- Daha sonra ikinci eleman ile ikinci en küçük elemanı yer değiştir.
  - İkinci elemandan sona kadar karşılaştırma yapılır.
- Bu işlem n-1 defa tekrar eder.

Seçmeli Sıralama Algoritması

- Selection Sort
- **Örnek:** 9, 5, 8, 3, 1 rakamlarının artan şekilde sıralanmasını seçmeli sıralama algoritmasıyla gerçekleştirelim. (Küçük olanı bul)
  - 1. iterasyon:
    - **Karşılaştır**
      - **9 5 8 3 1** min 5
      - **9 5 8 3 1** min 5
      - **9 5 8 3 1** min 3
      - **9 5 8 3 1** **min 1**
    - **Değiştir**
      - **9 5 8 3 1**
      - **1 5 8 3 9**

Seçmeli Sıralama Algoritması

- Selection Sort
- **Örnek:** 9, 5, 8, 3, 1 rakamlarının azalan şekilde sıralanmasını seçmeli sıralama algoritmasıyla gerçekleştirelim. (Küçük olanı bul)
  - 2. iterasyon:
    - **Karşılaştır**
      - **1 5 8 3 9** min 5
      - **1 5 8 3 9** min 3
      - **1 5 8 3 9** **min 3**
    - **Değiştir**
      - **1 5 8 3 9**
      - **1 3 8 5 9**

Seçmeli Sıralama Algoritması

- Selection Sort
- **Örnek:** 9, 5, 8, 3, 1 rakamlarının azalan şekilde sıralanmasını seçmeli sıralama algoritmasıyla gerçekleştirelim. (Küçük olanı bul)
  - 3. iterasyon:
    - **Karşılaştır**
      - **1 3 8 5 9** min 5
      - **1 3 8 5 9** **min 5**
    - **Değiştir**
      - **1 3 8 5 9**
      - **1 3 5 8 9**

Seçmeli Sıralama Algoritması

- Selection Sort
- **Örnek:** 9, 5, 8, 3, 1 rakamlarının azalan şekilde sıralanmasını seçmeli sıralama algoritmasıyla gerçekleştirelim. (Küçük olanı bul)
  - 4. iterasyon (son) :
    - **Karşılaştır**
      - **1 3 5 8 9 min 8**
    - **Değiştir**
      - **1 3 5 8 9**

Seçmeli Sıralama Algoritması

- Selection Sort
- **Örnek:** Algoritmayı kullanarak aşağıdaki verileri küçükten büyüğe sıralayalım.
  - Veriler : 25 57 48 37 12 92 86 33
  - Tekrar 1: **12** 57 48 37 **25** 92 86 33
  - Tekrar 2 : 12 **25** 48 37 **57** 92 86 33
  - Tekrar 3 : 12 25 **33** 37 57 92 86 **48**
  - Tekrar 4 : 12 25 33 **37** 57 92 86 48
  - Tekrar 5 : 12 25 33 37 **48** 92 86 **57**
  - Tekrar 6 : 12 25 33 37 48 **57** 86 **92**
  - Tekrar 7 : 12 25 33 37 48 57 **86** 92

Seçmeli Sıralama Algoritması

- Selection Sort
- Kodlar:

```
for (i = 0; i < n - 1; i++) {
    min = i;
    for (j = i + 1; j < n; j++) {
        if (dizi[j] < dizi[min]) {
            min = j;
        }
    }
    tmp = dizi[i];
    dizi[i] = dizi[min];
    dizi[min] = tmp;
}
```

Seçmeli Sıralama Algoritması

- Selection Sort
- Selection Sort da Bubble Sort gibi sıralama teknikleri içinde anlaşılması ve programlanması kolaydır. Ama zaman karmaşıklığı nedeniyle büyük verilerde etkili değildir.

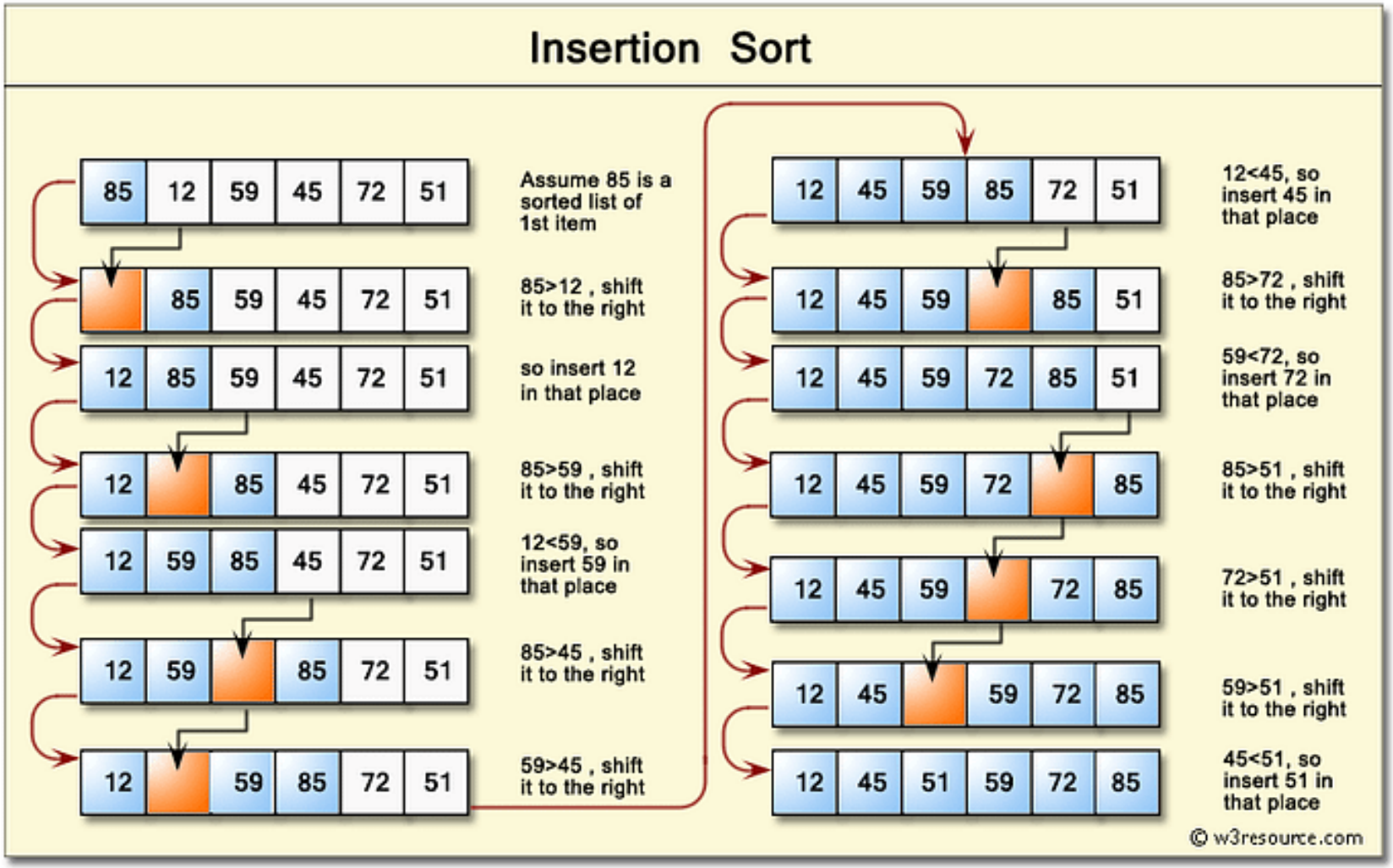
- Zaman Karmaşıklığı (Time Complexity):
  - En Kötü Durum (Worst Case): $O(n^2)$
  - Ortalama Durum (Average Case): $O(n^2)$
  - En İyi Durum (Best Case): $O(n^2)$
    - (Dizi sıralı olsa bile, algoritma yine her elemanı karşılaştırarak minimumu bulur.)
- Bellek Karmaşıklığı (Space Complexity): $O(1)$

Eklemeli Sıralama Algoritması

- Insertion Sort
- Insertion Sort, bir sıralama algoritmasıdır ve genellikle küçük veri setleri için uygundur.
- Algoritmanın temel mantığı, diziyi iki bölüme ayırmak (sıralı ve sıralanmamış kısımlar) ve sıralanmamış elemanları, sıralı kısımda doğru konumlarına yerleştirmektir. Bu işlem, kart oyunlarında kağıtları sıralamaya benzer.
- Nasıl Çalışır?
  - İlk eleman sıralı kabul edilir.
  - Diğer elemanlar teker teker ele alınır.
  - Her eleman sıralı kısımda doğru yerine yerleştirilir.
  - Bu işlem, tüm elemanlar işlenene kadar tekrarlanır.

Eklemeli Sıralama Algoritması

- Insertion Sort



Eklemeli Sıralama Algoritması

- Insertion Sort
- **Örnek:** Aşağıdaki verileri küçükten büyüğe sıralayalım.
  - Veriler : 56 38 21 83 11 67 43
  - Tekrar 1: 38 56 21 83 11 67 43
  - Tekrar 2 : 21 38 56 83 11 67 43
  - Tekrar 3 : 21 38 56 83 11 67 43
  - Tekrar 4 : 11 21 38 56 83 67 43
  - Tekrar 5 : 11 21 38 56 67 83 43
  - Tekrar 6: 11 21 38 43 56 67 83

Eklemeli Sıralama Algoritması

- Insertion Sort
- Kodlar:

```
for (i = 1; i < n; i++) {
    tmp = dizi[i];
    j = i - 1;
    while (tmp < dizi[j] && j >= 0) {
        dizi[j + 1] = dizi[j];
        j--;
    }
    dizi[j + 1] = tmp;
}
```

Eklemeli Sıralama Algoritması

- Insertion Sort
- Zaman Karmaşıklığı (Time Complexity):
  - En Kötü Durum (Worst Case): $O(n^2)$
  - Ortalama Durum (Average Case): $O(n^2)$
  - En İyi Durum (Best Case): $O(n)$
    - (Dizi sıralı olsa bile, algoritma yine her elemanı karşılaştırarak minimumu bulur.)
- Bellek Karmaşıklığı (Space Complexity): $O(1)$

Bubble, Selection, Insertion Karşılaştırma

Özellik	Bubble Sort	Selection Sort	Insertion Sort
Zaman Karmaşıklığı			
En Kötü Durum	$O(n^2)$	$O(n^2)$	$O(n^2)$
Ortalama Durum	$O(n^2)$	$O(n^2)$	$O(n^2)$
En İyi Durum	$O(n)$ (optimizasyonla)	$O(n^2)$	$O(n)$
Bellek Karmaşıklığı	$O(1)$	$O(1)$	$O(1)$
Kararlılık	Evet	Hayır	Evet
Yerinde Sıralama	Evet	Evet	Evet
Temel İşlem	Elemanları takas eder	Minimum elemanı seçer	Doğru yere ekler
Avantajlar	Basit ve kolay anlaşılır, optimizasyon yapılabilir	Daha az takas gerektirir	Sıralı dizilerde hızlıdır
Dezavantajlar	Yavaş, büyük veri için verimsiz	Çok fazla karşılaştırma yapar	Büyük veri setlerinde yavaş

Bubble, Selection, Insertion Karşılaştırma

- Selection Sort, az elemanlı veri grupları için (atamaların süresi çok fazla olmaz) ve karşılaştırmaların daha az yük getireceği basit anahtarlı durumlarda uygundur.
- Tam tersi için, insertion sort uygundur. Elemanlar bağlı listedelerse araya eleman eklemelerde veri kaydırma olmayacağından insertion sort mantığı uygundur.
- n'in büyük değerleri için quicksort, insertion ve selection sort'tan daha etkindir. Quicksort'u kullanmaya başlama noktası yaklaşık 30 elemanlı durumlardır; daha az elemanın sıralanması gerektiğinde insertion sort kullanılabilir.

Hızlı Sıralama Algoritması

- Quick Sort
- **Quick Sort** (Hızlı Sıralama), **böl ve fethet** (divide and conquer) yaklaşımını kullanan, oldukça verimli bir sıralama algoritmasıdır. Temel mantığı, bir **pivot** (dönme noktası) eleman seçip, diziyi bu pivota göre iki alt parçaya ayırmaktır:
- **Pivotun solunda**: Pivottan küçük elemanlar.
- **Pivotun sağında**: Pivottan büyük elemanlar.
- Alt parçalar aynı işlemle tekrar tekrar sıralanır, böylece dizi tamamıyla sıralanmış hale gelir.

Hızlı Sıralama Algoritması

- Quick Sort Nasıl Çalışır?
- **1. Pivot Seçimi**
- Dizinin bir elemanı pivot olarak seçilir. Bu, ilk eleman, son eleman, rastgele bir eleman ya da orta eleman olabilir.
- **2. Diziyi Bölme**
- Dizi, pivot elemanına göre iki alt parçaya ayrılır:
- Pivot elemanından **küçük olanlar** pivotun soluna,
- Pivot elemanından **büyük olanlar** pivotun sağına.
- Pivot, sıralı dizideki doğru konumuna yerleşir.

Hızlı Sıralama Algoritması

- Quick Sort Nasıl Çalışır?
- **3. Rekürsif Sıralama**
- Sol ve sağ alt parçalar için aynı işlem rekürsif olarak uygulanır.
- **4. Durdurma Şartı**
- Alt parçalar tek bir eleman kalana kadar bölünmeye devam eder. Tek elemanlı diziler zaten sıralı olduğu için bu noktada işlemler tamamlanır.

Hızlı Sıralama Algoritması

- Quick Sort
- Örnek:

							pivot
low							**high**
23	398	34	57	67	55	320	**100**
23	398	34	57	67	55	320	**100**
23	398	34	57	67	55	320	**100**
23	34	398	57	67	55	320	**100**
23	34	57	398	67	55	320	**100**
23	34	57	67	398	55	320	**100**
23	34	57	67	55	398	320	**100**
23	34	57	67	55	**100**	320	398

Hızlı Sıralama Algoritması

- Quick Sort
- Örnek:

				pivot2			
low2				**high2**			
23	34	57	67	**55**	**100**	320	398
23	34	57	67	**55**	**100**	320	398
23	34	57	67	**55**	**100**	320	398
23	34	57	67	**55**	**100**	320	398
23	34	**55**	67	57	**100**	320	398

Hızlı Sıralama Algoritması

- Quick Sort
- Örnek: **pivot3**

low3	**high3**						
23	**34**	**55**	67	57	**100**	320	398
23	**34**	**55**	67	57	**100**	320	398

Hızlı Sıralama Algoritması

- Quick Sort
- Zaman Karmaşıklığı (Time Complexity):
  - En Kötü Durum (Worst Case): $O(n^2)$
  - Ortalama Durum (Average Case): $O(n \log n)$
  - En İyi Durum (Best Case): $O(n \log n)$
- Bellek Karmaşıklığı (Space Complexity): $O(\log n)$

- Büyük veri setleri için ideal bir sıralama algoritmasıdır.
- En kötü durumun önlenmesi için genellikle rastgele pivot seçimi gibi stratejiler kullanılır.

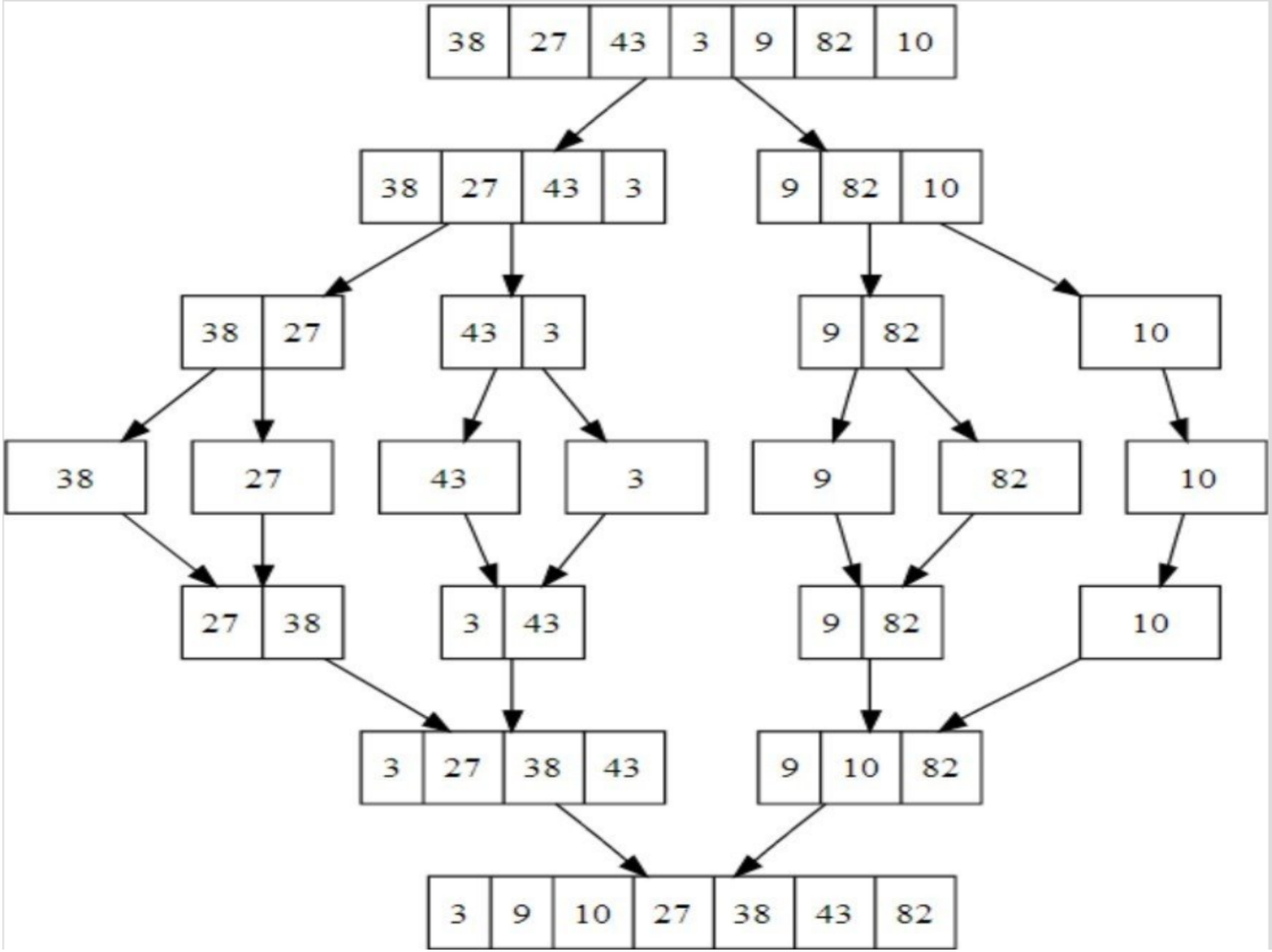
Birleştirmeli Sıralama Algoritması

- Merge Sort
- **Merge Sort** (Birleştirmeli Sıralama), **böl ve fethet** (divide and conquer) prensibini kullanan etkili bir sıralama algoritmasıdır.
- Algoritmanın temel amacı, diziyi tekrar tekrar küçük parçalara bölmek ve bu parçaları sıralayarak birleştirmektir.

Birleştirmeli Sıralama Algoritması

- Merge Sort
- Çalışma Mantığı:
- **1. Diziyi Bölme**
- Dizi, orta noktasından iki alt parçaya bölünür. Bu işlem, alt dizilerin uzunluğu 1 olana kadar rekürsif olarak devam eder. Uzunluğu 1 olan diziler zaten sıralı kabul edilir.
- **2. Birleştirme**
- Alt diziler sıralı bir şekilde birleştirilerek daha büyük sıralı alt diziler oluşturulur. Bu işlem, tüm dizi sıralanana kadar devam eder.

Birleştirmeli Sıralama Algoritması



Birleştirmeli Sıralama Algoritması

- Merge Sort
- **Örnek:** 5,4,2,9,6,1,3,7 rakamlarının artan şekilde sıralanmasını birleştirmeli sıralama algoritmasıyla gerçekleştirelim.
  - 1. **Diziyi Bölme**

    5,4,2,9,6,1,3,7 → 5,4,2,9 ve 6,1,3,7

    5,4,2,9 → 5,4 ve 2,9

    5,4 → 5 ve 4

    Benzer şekilde diğer parçalar da aynı şekilde bölünür.
  - **2. Birleştirme**

    5 ve 4 → 4,5

    2 ve 9 → 2,9

    4,5 ve 2,9 → 2,4,5,9

    Son olarak, tüm alt diziler birleşip 1,2,3,4,5,6,7,9 olur.

Birleştirmeli Sıralama Algoritması

- Merge Sort
- Zaman Karmaşıklığı (Time Complexity):
  - En Kötü Durum (Worst Case): O(O(n log n)
  - Ortalama Durum (Average Case): O(n log n)
  - En İyi Durum (Best Case): O(n log n)
- Bellek Karmaşıklığı (Space Complexity): O(n)

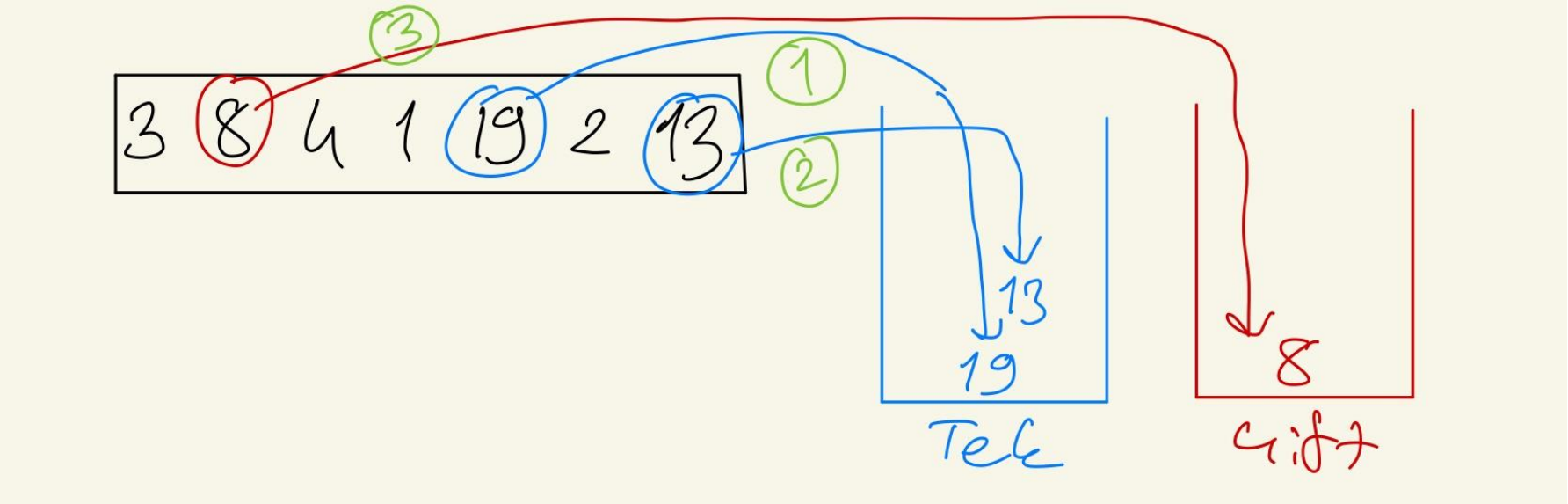
- Büyük veri setleri ve kararlı sıralamanın önemli olduğu durumlarda tercih edilen bir algoritmadır.
- En kötü durumda Quicksort'a ($O(n^2)$) göre daha avantajlıdır.
- Fakat ekstra bellek kullanımı nedeniyle büyük veri kümelerinde yer problemi yaşanabilir.

Özellik	Bubble Sort	Selection Sort	Insertion Sort	Quick Sort	Merge Sort
Zaman Karmaşıklığı	$O(n^2)$	$O(n^2)$	$O(n^2)$ (Kötü durum)	Ortalama: $O(n \log n)$	$O(n \log n)$
En İyi Durum	$O(n)$	$O(n^2)$	$O(n)$	$O(n \log n)$	$O(n \log n)$
En Kötü Durum	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(n \log n)$
Bellek Karmaşıklığı	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$	$O(n)$
Kararlılık	Kararlı	Kararsız	Kararlı	Kararsız	Kararlı
Çalışma Prensibi	Komşu elemanları değiştirerek sıralar	En küçük elemanı seçer	Yerleştirme prensibi	Böl ve fethet	Böl ve birleştir
Avantajlar	Basit ve küçük veri setlerinde etkili	Basit ve ek bellek gerektirmez	Küçük/sıralı verilerde hızlı	Büyük veri setlerinde hızlı ve etkili	Her durumda kararlı ve hızlı
Dezavantajlar	Büyük veri setlerinde yavaş	Kararsız, yavaş	Büyük veri setlerinde yavaş	Kötü pivot seçilirse yavaş ($O(n^2)$)	Ek bellek kullanımı yüksek

Sıralama Algoritmaları Performansı

- Aşağıdaki çevrim içi araçla algoritmaların başarılarını farklı senaryolarda izleyebilirsiniz. (https://www.toptal.com/developers/sorting-algorithms)

  - Sol taraftaki oynatmalar tıklanırsa, rastgele liste, neredeyse sıralı liste, ters sıralı liste, bazıları eşit listelerde algoritma başarılarını gözlemleyin.
  - Veya Üst taraftaki ilgili algoritmaya tıklayarak farklı senaryolardaki başarısını gözlemleyebilirsiniz. Örneğin, tüm senaryolarda nlogn zaman gerektiren merge sıralamayı gözlemleyiniz.

Örnek Uygulamalar

1. Verilen bir dizideki elemanların her bir adımda en büyüğünü tespit edip (Kabarcık sıralama kullanarak) Çift veya Tek sayı olmasına göre ilgili bir yığına (stack)'a atan kodu yazıp işlem sonunda her iki stack i de yazdırınız.